Unit 3- Part 1

Array in C

By: **Rishi Kumar**

Multidimensional Array

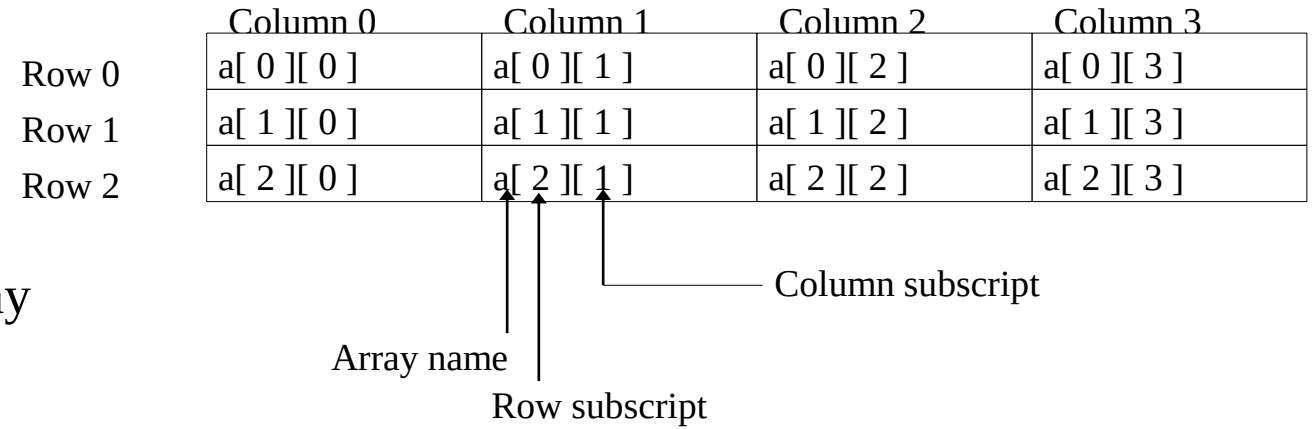
- Arrays with more than one index: number of dimensions = number of indexes
- Arrays with more than two dimensions are a simple extension of two-dimensional (2-D) arrays.
- A **2-D array** corresponds to a **collection of components, all of the same type, structured in two dimensions**:
  - one dimension is the **row**
  - the other dimension is the **column**
  - **cell:** an intersection of a row and column
  - an array element corresponds to a cell in the table
- Syntax:
- <**type**> <**arrayName**>**[**<**row_size**>**] [**<**column_size**>**]**
- **Example: int** Ar[10][5]; **float** matrixNum[20][25];
- The array elements are all values of the type **<type>.**
- The size of the array is indicated by **<array_size>**, the number of elements in the array.
- **<array_size>** must be an **int** constant or a constant expression.
- Note that an array can have multiple dimensions.

2D Array Representation

- The number of subscripts determines the *dimensionality* of an array.
- For example, x[i] refers to an element of a one-dimensional array, x. Similarly, y[i][j] refers to an element of a two-dimensional array, y, and so on.
  - int b[3][5]={{1,2,3,4,5},{6,7,8,9,10},{11,12,13,14,15}};
- **Unsized Array Initializations**
  - C compiler automatically creates an array big enough to hold all the initializers. This is called an unsized array.
  - The following are examples of declarations with initialization.
    - **char e1[] =“read error\n”;**
    - **char e2[] =“write error\n”;**
- Multi-dimensional arrays are kept in computer memory as a linear sequence of variables.
- The elements of a multi-dimensional array are stored contiguously in a block of computer memory.
- The number of subscripts determines the *dimensionality* of an array.
- The separation of initial values into rows in the declaration statement is not necessary.
- If unsized arrays are declared, the C compiler automatically creates an array big enough to hold all the initializers.

Memory Representation

- Multiple subscripts
  - **a[i][j]**
  - Tables with rows and columns
  - Specify row, then column
  - "Array of arrays"
    - **a[0]** is an array of 4 elements
    - **a[0][0]** is the first element of that array
- To initialize
  - Default of **0**
  - Initializers grouped by row in braces



int b[2][2] = { { 1, 2 }, { 3, 4 } };

Row 0 Row 1

1	2
3	4

int b[2][2] = { 1, 0, 3, 4 };

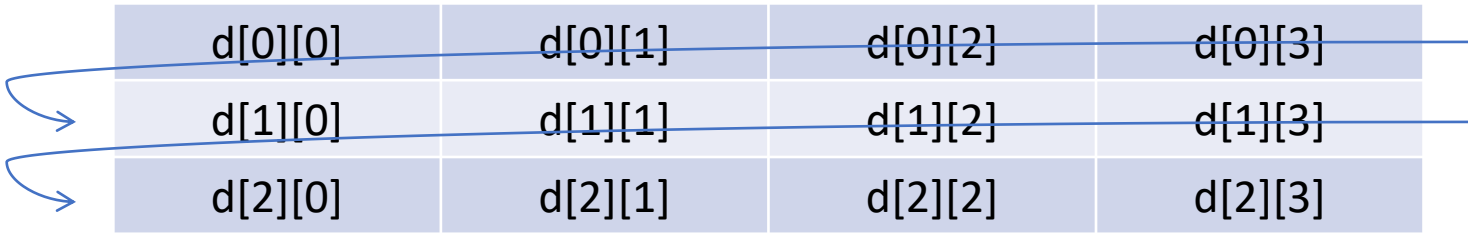
1	0
3	4

Memory Allocation for Two-Dimensional Arrays

- Storage for all array elements is determined at compile time.
- The memory used to hold an array is required the entire time that the array is in existence.
- The following formula determines the number of bytes of memory that will be allocated:

  bytes = rows * columns * number_of_bytes_in_type

- For example, an integer array (with two-byte integers) with dimensions 100,100 would require 100 * 100 * 2 = 20,000 bytes.



```c
#include <stdio.h>

int main()
{
   int i, j, m, n;
   int matrix[10][20];

   printf("Enter number of rows : ");
   scanf("%d", &m);
   printf("Enter number of columns : ");
   scanf("%d", &n);

   /* Input data in matrix */
   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         printf("Enter data in [%d][%d]: ", i, j);
         scanf("%d", &matrix[i][j]);
      }
   }

   /* Display the matrix */
   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         printf("%d\t", matrix[i][j]);
      }
      printf("\n");
   }

   return 0;
}
```

Processing a 2-D Array

We can generate the 2D array by using this program:

```
#include <stdio.h>
int main()
{
    int row=3, col=4;
    int matrix[row][col];
    for(row=0; row < 3; ++row)
          {
               for(col=0; col < 4; ++col)
               {
                     matrix[row][col] = (row*4)+ col +1;
                     printf(Matrix is: matrix[row][col] );
               }
  printf(‘\n’);
}
return(0);}
```

```
1234
5678
9101112
```

Why do we care about storage order?

- If you keep within the “paradigm” of the multidimensional array, the order doesn’t matter…
- But if you use tricks with pointer arithmetic, it matters a lot
- It also matters for initialization
  - To initialize d like this:
    - use this:

      int d[2][4] = {0, 1, 2, 3, 4, 5, 6, 7};
    - rather than this

      int d[2][4] = {0, 4, 1, 5, 2, 6, 3, 7};

0	1	2	3
4	5	6	7

ARRAYS OF STRINGS:
2D CHARACTER ARRAY

- A two-dimensional array of strings can be declared as follows:

  <data_type> <string_array_name>[<row_size>] [<columns_size>];

- Consider the following example on declaration of a two-dimensional array of strings.

  char s[5][30];

- Two-dimensional string arrays can be initialized as shown

  char s[5][10] ={“Cow”,”Goat”,”Ram”,”Dog”,”Cat”};

- which is equivalent to

  s[0] C o w \0

  S[1] G o a t \0

  S[2] R a m \0

  S[3] D o g \0

  S[4] C a t \0

- Here every row is a string. That is, s[i] is a string. Note that the following declarations are invalid.
  - char s[5][] ={“Cow”,“Goat”,“Ram”,”Dog”,“Cat”};
  - char s[][] ={“Cow”,“Goat”,“Ram”,“Dog”,“Cat”};

Multidimensional Arrays as parameters

- Only the first subscript may be left unspecified

```
void f(int matrix[][10]);      /* OK */
void g(int (*matrix)[10]);     /* OK */
void h(int matrix[][]);                /* not OK */
```

- Why?
  - Because the other sizes are needed for scaling when evaluating subscript expressions (see slide 10)
  - This points out an important drawback to C:
    - Arrays do not carry information about their own sizes!
    - If array size is needed, you must supply it somehow
    - (e.g., when passing an array argument, you often have to pass an additional "array size" argument) – bummer

Summary about Arrays

- An array may be thought of as a collection of variables, all of the same type.
- An array is also may be thought of as a single object with a large composite value of all the elements of the array.
- Arrays are objects created with *new* in a manner similar to objects discussed previously.
- Array indexes use zero-numbering:
  - they start at 0, so index *i* refers to the*(i+1)th* element;
  - the index of the last element is (length-of-the-array - 1).
  - Any index value outside the valid range of 0 to length-1 will cause an **array index out of bounds error** when the program *runs*.
- A method may return an array.
- A "partially filled array" is one in which values are stored in an initial segment of the array:
  - use an int variable to keep track of how many variables are stored.

Summary about Arrays

- An array element can be used as an argument to a method any place the base type is allowed:
  - if the base type is a primitive type, the method cannot change the array element;
  - if the base type is a class, the method *can* change the array element.
- When you want to store two or more different values (possibly of different data types) for each index of an array,
  - parallel arrays (multiple arrays of the same length)
  - use a class that have multiple types/values.
- An accessory method that returns an array corresponding to a private instance variable of an array type should be careful to return a copy of the array, and not return the private instance variable itself.
- Arrays can have more than one index. Each index is called a *dimension*.
- Hence, *multidimensional* arrays have multiple indexes,
  - e.g. an array with two indexes is a two-dimensional array.
- A two-dimensional array can be thought of as a grid or table with rows and columns:
  - one index is for the row, the other for the column.

Matrices are widely used in various fields such as physics, engineering, and computer science. In C programming language, matrices are used to represent and manipulate multi-dimensional arrays of data. Here are a few examples of why we might need to use matrices in C:

Image Processing: Matrices are commonly used to represent images in computer vision and image processing applications. Each element of the matrix represents the intensity of a pixel in the image, and the matrix can be used to apply various image processing algorithms such as filtering, transformation, and compression.

Linear Algebra: Matrices are used to represent and manipulate linear equations, which are widely used in physics, engineering, and computer graphics. Matrices can be used to solve systems of linear equations, calculate determinants, and perform matrix operations such as inversion, transposition, and multiplication.

Machine Learning: Matrices are used to represent data in machine learning algorithms. Matrices can be used to represent feature vectors, training sets, and weight matrices in neural networks.

Game Development: Matrices are used to represent and manipulate 3D graphics in game development. Matrices can be used to represent transformations such as translation, rotation, and scaling of objects in a 3D scene.

Robotics and Control systems: Matrices are used to represent and manipulate the state of a system in robotics and control systems, matrices can be used to represent the position, velocity, and acceleration of a robot's joints, or to represent the state of a control system.

Addition of Matrix

```
#include <stdio.h>
int main() {
   int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
   int b[3][3] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
   int c[3][3];

   int i, j;
   for (i = 0; i < 3; i++) {
      for (j = 0; j < 3; j++)
{
         c[i][j] = a[i][j] + b[i][j];
      } }
   printf("Result of addition: \n");
   for (i = 0; i < 3; i++) {
      for (j = 0; j < 3; j++)
{
         printf("%d ", c[i][j]);
      }
      printf("\n");
   }
   return 0;  }
```

```c
#include <stdio.h>
int main() {
  int r, c, a[100][100], b[100][100], sum[100][100], i, j;
  printf("Enter the number of rows (between 1 and 100): ");
  scanf("%d", &r);
  printf("Enter the number of columns (between 1 and 100):
");
  scanf("%d", &c);

  printf("\nEnter elements of 1st matrix:\n");
  for (i = 0; i < r; ++i)
    for (j = 0; j < c; ++j) {
      printf("Enter element a%d%d: ", i + 1, j + 1);
      scanf("%d", &a[i][j]);
    }

  printf("Enter elements of 2nd matrix:\n");
  for (i = 0; i < r; ++i)
    for (j = 0; j < c; ++j) {
      printf("Enter element b%d%d: ", i + 1, j + 1);
      scanf("%d", &b[i][j]);
    }

  // adding two matrices
  for (i = 0; i < r; ++i)
    for (j = 0; j < c; ++j) {
      sum[i][j] = a[i][j] + b[i][j];
    }

  // printing the result
  printf("\nSum of two matrices: \n");
  for (i = 0; i < r; ++i)
    for (j = 0; j < c; ++j) {
      printf("%d   ", sum[i][j]);
      if (j == c - 1) {
        printf("\n\n");
      }
    }

  return 0;
}
```

Subtraction of Matrix

```
#include < stdio.h >
int main()
{
    int m, n, c, d, first[10][10], second[10][10], difference[10][10];
    printf("Enter the number of rows and columns of matrix\n");
    scanf("%d%d", & m, & n);
    printf("Enter the elements of first matrix\n");
    for (c = 0; c < m; c++)
        for (d = 0; d < n; d++) scanf("%d", & first[c][d]);
    printf("Enter the elements of second matrix\n");
```

```
for (c = 0; c < m; c++)
        for (d = 0; d < n; d++) scanf("%d", & second[c][d]);
    printf("Difference of entered matrices:-\n");
    for (c = 0; c < m; c++)
    {
        for (d = 0; d < n; d++)
        {
            difference[c][d] = first[c][d] - second[c][d];
            printf("%d\t", difference[c][d]);
        }
        printf("\n");
    }
    return 0;
}
```

Multiplication of Matrix

```c
#include<stdio.h>
#include<stdlib.h>
int main(){
int a[10][10],b[10][10],mul[10][10],r,c,i,j,k;
system("cls");
printf("enter the number of row=");
scanf("%d",&r);
printf("enter the number of column=");
scanf("%d",&c);
printf("enter the first matrix element=\n");
for(i=0;i<r;i++)
{
for(j=0;j<c;j++)
{
scanf("%d",&a[i][j]);
}
}
printf("enter the second matrix element=\n");
for(i=0;i<r;i++)
{
for(j=0;j<c;j++)
{
scanf("%d",&b[i][j]);
}  }
```

```c
printf("multiply of the matrix=\n");
for(i=0;i<r;i++)
{
for(j=0;j<c;j++)
{
mul[i][j]=0;
for(k=0;k<c;k++)
{
mul[i][j]+=a[i][k]*b[k][j];
}
}
}
//for printing result
for(i=0;i<r;i++)
{
for(j=0;j<c;j++)
{
printf("%d\t",mul[i][j]);
}
printf("\n");
}
return 0;
}
```

Example: Transpose of a matrix

```
#include <stdio.h>
int main()
{
  int row,col;
  int i, j, value;
  int mat[10][10], transp[10][10];
  printf("\n Input the number of rows:");
  scanf("%d", &row);
  printf("Input number of cols:");
  scanf("%d", &col);
  for(i = 0 ; i < row; i++)
  {
    for(j = 0 ; j < col; j++)
    printf("Input Value for : %d: %d:",
           i+1,j+1);
    scanf("%d", &value);
    mat[i][j] = value;
  }
}
printf("\n Entered Matrix is as follows:\n'
for(i = 0; i < row; i++)
{
  for(j = 0; j < col; j++)
  {
    printf("%d", mat[i][j]);
  }
    printf("\n");
  }
  for(i = 0; i< row; i++)
  {
  for(j = 0; j < col; j++)
  {
   transp[i][j]= mat[j][i];
  }
  }
   printf("\n Transpose of the matrix is as\
          follows:\n");
   for(i = 0; i < col; i++)
   {
     for(j = 0; j < row; j++)
    {
     printf("%d", transp[i][j]);
     }
     printf("\n");
    }
      return 0;
    }
```

```c
/* C program to search element in array */
#include <stdio.h>

#define MAX_SIZE 100  // Maximum array size
int main()
{
   int arr[MAX_SIZE];
   int size, i, toSearch, found;
   /* Input size of array */
   printf("Enter size of array: ");
   scanf("%d", &size);
   /* Input elements of array */
   printf("Enter elements in array: ");
   for(i=0; i<size; i++)
   {
      scanf("%d", &arr[i]);
   }
   printf("\nEnter element to search: ");
   scanf("%d", &toSearch);
/* Assume that element does not exists in array */
   found = 0;
for(i=0; i<size; i++)
   {
      /*  If element is found in array then raise found flag
       * and terminate from loop.*/
      if(arr[i] == toSearch)
      {
         found = 1;
         break;
      }
   }

   /* If element is not found in array*/
   if(found == 1)
   {
  printf("\n%d is found at position %d", toSearch, i + 1);
   }
   else
   {
      printf("\n%d is not found in the array", toSearch);
   }
   return 0;
}
```

/ C program to accept N numbers and arrange them in an ascending order*/**

```
#include <stdio.h>
void main()
{

    int i, j, a, n, number[30];
    printf("Enter the value of N \n");
    scanf("%d", &n);

    printf("Enter the numbers \n");
    for (i = 0; i < n; ++i)
        scanf("%d", &number[i]);
```

```
for (i = 0; i < n; ++i)
      {
          for (j = i + 1; j < n; ++j)
         {
             if (number[i] > number[j])
            {
                a =  number[i];
               number[i] = number[j];
               number[j] = a;
            }
          }
       }
printf("The numbers arranged in ascending order are
given below \n");
       for (i = 0; i < n; ++i)
          printf("%d\n", number[i]);
   }
```

Assignment including Lab Practical:
Submission after Dussehra in Lab Sessions

1. Write a program in C to read n number of values in an array and display them in reverse order.
2. Write a program in C to find the sum of all elements of the array.
3. Write a program in C to copy the elements of one array into another array.
4. Write a program in C to count the total number of duplicate elements in an array.
5. Write a program in C to find the maximum and minimum elements in an array.
6. Write a C program to sort the elements of an array in descending order.
7. Write a program in C to separate odd and even integers into separate arrays.
8. Write a program in C to merge two arrays of the same size sorted in descending/ascending order.
9. Write a program in C to merge two arrays of the same size sorted in descending order.
10. WAP using Switch case: Consider two matrices of the size m and n. Implement matrix operation and display. Show these things in program 1) Read matrix elements and display 2) Matrix Multiplication and display 3) addition of matrix and display 4)Subtraction of Matrix and display 5)Transpose of Matrix and display.